

课程笔记

课程笔记：[Modern Agent] 16 Claude Code 记忆（Memory）系统 设计，记忆语义分类，记忆召回，记忆 写入、更新与维护

五道口纳什

2026 年 4 月 10 日

Memory System 记忆 Claude Code

视频作者/频道：五道口纳什

发布日期：2026 年 4 月 10 日

视频时长：21:37

视频链接：<https://www.bilibili.com/video/BV1ZA93BtEKW/>

目录

1 引入与整体动机	2
1.1 课程背景与研究契机	2
1.2 三大核心线索：分类、落盘与召回	2
1.2.1 记忆分类：taxonomy 的设计必要性	2
1.2.2 本地持久化：manifest 与 topic 文件	3
1.2.3 召回与 wire layer 注入	3
1.3 语言模型驱动的记忆系统与幻觉防范	4
1.4 本章小结	4
2 文件结构与 Manifest 设计	4
2.1 Manifest 的角色：轻量级目录摘要	4
2.2 topic.md：真正承载记忆的文件	6
2.3 front matter 的作用	6
2.4 本章小结	7
3 四种语义类型详解	7
3.1 User：用户画像	8
3.2 Feedback：行为约束与规则	8
3.3 Project：代码之外的现实	9
3.4 Reference：去哪找事实	9
3.5 四类 memory 对比	10
3.6 本章小结	10
4 写人与召回机制	11
4.1 Extract Memory Agent：turn 结束后的持久化	11
4.1.1 什么是 turn	11
4.1.2 extract agent 的工作流程	11
4.2 Selecting Memory Agent：基于 manifest 的语义召回	12
4.2.1 召回的本质	12
4.2.2 召回的核心限制	12
4.2.3 限制背后的设计考量	13
4.3 本章小结	13
5 Prompt 注入、Memory Policy 与更新维护	13
5.1 Memory Policy 与 System Prompt	13
5.2 幻觉防范：memory 不是 source of truth	14
5.3 注入方式：system prompt、user context 与 nested memory	15
5.4 更新维护机制：extract sub-agent 的调用时机与 system prompt 继承	15
5.5 本章小结	16
6 总结与延伸	16
6.1 本章小结	17

1 引入与整体动机

1.1 课程背景与研究契机

在 Claude Code 源码泄露之后，我们终于有机会深入窥探一个现代 coding agent 是如何设计与实现其 memory 系统的。memory 的本质作用，是让 agent 对”用户是谁”、”项目现状如何”、”之前学到了什么”等问题保持长期认知，从而在多次交互中表现得越来越”熟门熟路”。然而，memory 的存储、分类、召回与注入都不是简单的 KV 缓存，而是一个需要与语言模型能力深度耦合的工程体系。

本节聚焦于整个 memory 系统的宏观动机与三条核心分析线索：

- **分类 (taxonomy)**：auto memory 按语义被严格划分为哪几类？
- **落盘 (持久化与维护)**：记忆如何被写入本地文件系统并持续更新？
- **召回与注入 (wire layer)**：记忆如何从文件中被提取，并以何种方式注入到 agent loop 的 API 请求中？

理解这三条线索，就能建立对后续章节（manifest 结构、四种语义类型、selecting/extracting agent、幻觉防范）的整体认知框架。

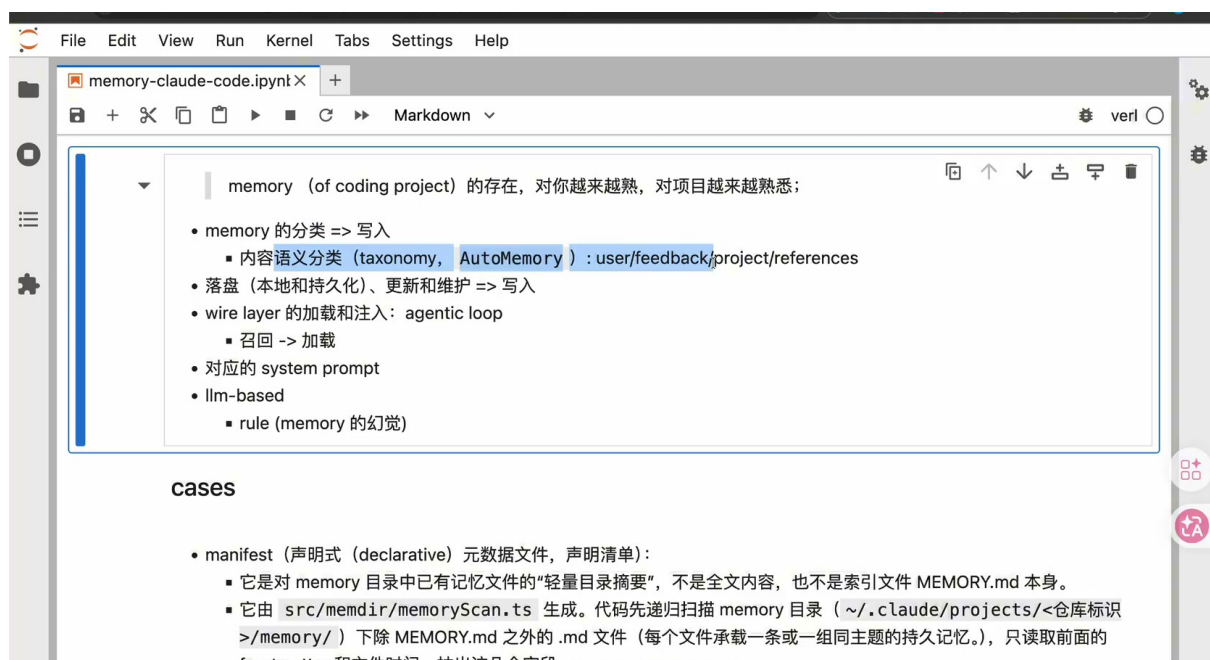


图 1: Claude Code memory 的分类体系总览，涵盖了内容语义分类、落盘机制、wire layer 注入方式、system prompt 规则约束以及 memory 幻觉防范等核心概念。¹

1.2 三大核心线索：分类、落盘与召回

1.2.1 记忆分类：taxonomy 的设计必要性

不同于随意堆叠的文本片段，Claude Code 的 auto memory 对内容语义施加了强约束。在源码中，这一约束被定义在 `memory-types.ts` 中，将自动维护的长期记忆划分为四类：

¹ 视频画面时间区间：00:02:30–00:02:35。

- **user**: 关于”用户是谁”的记忆，影响回答风格、解释粒度与默认协作方式；
- **feedback**: 对 agent 以后该怎么做的行为约束，既记录负反馈也记录正反馈；
- **project**: 无法从代码库直接推导出的项目上下文（如截止日期、merge 冻结期、重构原因）；
- **reference**: 不存事实本身，而是”去哪里找事实”的指向性信息（如 Linear project、Slack channel）。

taxonomy 的定义

taxonomy 是 auto memory 的语义分类体系。Claude Code 强制要求所有自动维护的长期记忆必须落入上述四类之一。这种分类不是装饰，而是直接决定了：提取 agent 应该如何撰写正文、召回 agent 应该如何判断相关性、以及最终注入 agent loop 时应该扮演什么角色。

1.2.2 本地持久化：manifest 与 topic 文件

auto memory 是一个**文件型知识库**，它的核心落盘结构包含两类文件：

- **memory.md (manifest)**: 一种轻量级目录摘要，记录各 topic 的名称与简短描述，用于渐进式加载与召回，但**不是全文内容，也不是严格索引**；
- **各 topic.md 文件**: 真正承载持久化记忆的正文文件，每个文件围绕一条或一组相同主题，采用 topic-oriented 命名（如 `user-role.md`、`feedback-testing.md`）。

在读取时，系统只抽取 topic.md 文件的 front matter (`name`、`description`、`type`) 以及文件时间，从而在不加载全文的情况下完成初步筛选。这一设计保证了召回过程的轻量与高效。

```
1 manifest (memory.md)      -> 轻量目录摘要，用于快速定位相关 topic
2 topic files (*.md)       -> 真正持久化的记忆正文，按语义分类命名
```

Listing 1: manifest 与 topic 文件的职责边界

1.2.3 召回与 wire layer 注入

即使记忆已经正确分类并落盘，仍然需要回答一个关键问题：在每轮用户 query 到来时，哪些记忆应该被加载到 agent 的上下文中？这就涉及**召回**与**wire layer**两个概念。

- **召回 (selecting memory agent)**: 基于 manifest 中 topic 的名称与描述，判断哪些文件与当前 query 最相关，每次最多返回 5 个；只有”高度确信有用”的记忆才会被选中。
- **wire layer**: 将召回的记忆以 API 请求的形式发送给远端模型，对应于把相关上下文注入到 agent loop 的请求体中（如 ChatGPT/Codex API 的 request body）。

wire layer 的定义

wire layer 是将召回的记忆以 API 请求方式发送给远端模型的层面。在 Claude Code 的语境下，wire layer 对应于把 `relevant_memories` 注入到 agent loop 的请求体 (request body) 中，使远端模型在生成回复时能够使用这些上下文。

1.3 语言模型驱动的记忆系统与幻觉防范

Claude Code 的写入、加载与更新在很多时候都是由语言模型本身驱动的。例如：

- 每轮交互结束后，主 agent 会 fork 一个 **extract memory agent**，由它基于整轮对话内容决定“更新旧文件”还是“创建新文件”；
- 召回阶段，**selecting memory agent** 也要依赖语言模型来判断 manifest 中列出的 topic 是否与当前 query 相关。

正因为 memory 系统是由 LLM 驱动的，prompt 设计必须充分考虑**记忆幻觉（memory hallucination）**的问题。memory 中提到的具体符号（函数名、文件路径、flag）仅仅是历史线索，不等同于当前事实。代码库是持续演进的，而 memory 只是“写下它时的认知”。在下一章中我们会看到，system prompt 明确通过 policy 条款要求模型不要将 memory 视为 source of truth。

```
1 - memory 不是 source of truth
2 - 具体符号只能作为历史线索，必须回到当前代码库验证
3 - 真正给用户建议前，先读取并确认当前仓库状态
```

Listing 2: memory policy 的核心约束示例

1.4 本章小结

本章从 Claude Code 源码泄露这一契机出发，梳理了分析 memory 系统的三条主线：

1. **taxonomy**：auto memory 被强约束为 user、feedback、project、reference 四类语义；
2. **落盘机制**：以 memory.md 为 manifest（轻量摘要），以各 topic.md 为持久化正文，实现渐进式加载；
3. **召回与注入**：通过 selecting agent 基于 manifest 筛选相关记忆，再经 wire layer 注入到 API 请求体中。

此外，我们强调了 memory 系统的 LLM 驱动特性带来的设计挑战：prompt 中必须设置清晰的规则与幻觉防范策略，避免把过时的历史记忆误认为不可动摇的事实。这些铺垫将帮助我们在后续章节中更深入地理解 manifest 结构、四种语义类型的写作规范，以及 extract/select agent 的 prompt 设计。

2 文件结构与 Manifest 设计

在 Claude Code 的 auto memory 体系中，持久化记忆并非以单一文件的形式存储，而是采用了一种“索引-正文分离”的两层结构：轻量级的 memory.md 作为入口（即 manifest），而真正承载知识内容的则是分散在各个 topic 文件中的持久化记忆。本节将详细拆解这一设计的核心概念、文件结构以及 front matter 的作用。

2.1 Manifest 的角色：轻量级目录摘要

Manifest 是对 memory 目录中已有记忆文件的声明式元数据摘要。它的本质是一份“生命清单”，用于快速了解当前有哪些主题类别的记忆存在，而不是记忆的全文内容，也不是严格意义上的索引文件。

在 Claude Code 的实现中，`memory.md` 就扮演着 manifest 的角色。系统会周期性地扫描项目目录（通常是 `/.claude/projects/<project>/memory/`），收集除 `memory.md` 本身之外所有 `.md` 文件的关键信息，并将它们汇总到这个入口文件中。读取时，系统只抽取每个 `topic` 文件最前面的 front matter 以及文件的时间戳，生成一个轻量级的清单，方便后续的渐进式加载与召回。

这种设计与之前介绍 Cortex 技能系统时的做法有异曲同工之妙：在接收用户的第一条真实 query 之前，系统会预先注入一条元数据消息，其中包含相关技能的名称与描述。`memory.md` 同理，它为后续的 selecting memory agent 提供了一个“先看目录、再读正文”的入口，避免了在每次召回时都要加载全部记忆内容的高昂代价。

什么是声明式元数据文件？

声明式（Declarative）意味着文件描述的是“有什么”，而不是“怎么做”。manifest 不关心（`topic.md` 中）知识的具体生成逻辑或应用方式，它只陈述当前记忆库中存在哪些主题、它们属于什么类型、以及简短的描述。这种“描述即数据”的风格使得上层召回模块可以用极低的成本完成第一轮筛选。

manifest 不是全文，也不是严格索引

很多初学者容易把 `memory.md` 误解为记忆的完整副本，或者像数据库索引那样精确映射到每个字段。实际上，manifest 只是一个轻量级的目录摘要——它记录的是“有哪些 topic”和“它们大概是关于什么的”，真正的细节全部下沉到各个 `topic.md` 文件中。这种刻意保持“轻量”的设计，是为了让 LLM 在召回阶段能够快速判断相关性，而不被过量的正文内容淹没。

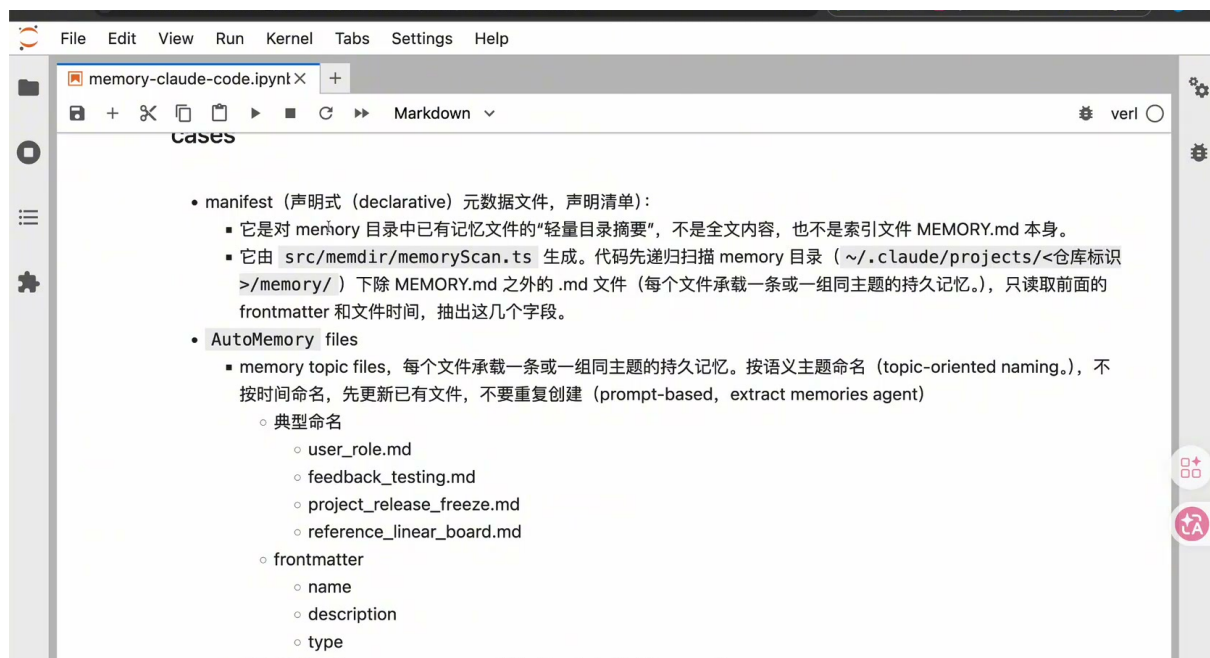


图 2: Manifest 声明与 AutoMemory 文件结构示意。图中展示了 manifest 作为轻量级元数据入口的定位，以及 topic-oriented 的命名规则与 front matter 字段。²

²视频画面时间区间：00:03:20–00:03:25。

2.2 topic.md: 真正承载记忆的文件

与 manifest 形成对照的是 **topic.md**——它们是真正承载持久化记忆正文的文件。manifest 告诉我们”有什么”，而 topic.md 告诉我们”具体内容是什么”。

auto memory 中的 topic 文件遵循以下几个关键设计原则：

1. **主题命名,而非时间命名**: 每个文件按语义主题命名(例如 `user-role.md`、`feedback-testing.md`、`project-release-freeze.md`)，而不是按创建时间或对话轮次编号。这保证了同一主题的记忆可以持续更新、追加，而不会随着对话轮数的增加而不断产生碎片化文件。
2. **每个文件承载一条或一组相同主题的记忆**: 相关性强的事实或规则可以聚合在同一个 topic 文件中，减少文件数量，提升召回效率。
3. **优先更新已有文件，避免重复创建**: 当 extract memory agent 在 turn 结束后写入新的记忆时，它会先扫描 manifest，判断当前是否已有相关主题的文件；如果有，则倾向于更新该文件而不是创建新文件。

topic 文件的命名通常会带上其所对应的语义类型后缀或前缀，以呼应 taxonomy 中的四大类别 (user、feedback、project、reference)。例如：

```
1 ~/.claude/
2   projects/
3     my-project/
4       memory/
5         memory.md          <-- manifest (轻量摘要)
6         user-role.md       <-- 用户画像
7         feedback-testing.md <-- 行为约束 (测试相关)
8         project-roadmap.md <-- 项目上下文
9         reference-linear.md <-- 外部参考指向
```

Listing 3: 典型的 Auto Memory 目录结构示例

2.3 front matter 的作用

每个 topic.md 文件的开头都包含一段 YAML 风格的 **front matter**，它与 skill 文件的风格非常相似。front matter 是系统快速识别和召回记忆的核心依据，其典型字段如下：

```
1 ---
2 name: user-role
3 description: 用户的职业背景、技术栈偏好以及对解释深度的期望
4 type: user
5 ---
```

Listing 4: topic.md 的 front matter 示例

其中三个字段各司其职：

- **name**: topic 的唯一标识名称，通常与文件名保持一致，用于在 manifest 中快速引用。
- **description**: 对该 topic 内容的简短描述，是 selecting memory agent 判断相关性的主要依据之一。

- **type**: 记忆的语义分类, 限定为四种类别之一——**user**、**feedback**、**project**、**reference**。这一字段不仅用于组织目录, 更决定了该记忆在注入 agent loop 时的处理策略与优先级。

系统在扫描目录时, 只会读取 front matter 和文件的时间戳, 而不会把整个文件内容都读入 manifest。这种“元数据优先”的方式, 使得在成百上千条记忆中进行首轮筛选成为可能。

front matter 与渐进式加载

front matter 的存在让 Claude Code 可以采用“渐进式加载”策略: 第一步, 通过读取 manifest 中的 name/description 快速判断哪些 topic 可能与当前用户 query 相关; 第二步, 只加载被判定为相关的少量 topic.md 的完整正文, 注入到 agent loop 中。这样有效控制了上下文长度, 避免 tokens 被不相关记忆挤占。

2.4 本章小结

Claude Code 的 auto memory 采用了一种清晰的两层文件结构: **memory.md** 作为 manifest, 是整个记忆库的轻量级入口; 而各个 topic.md 文件则是承载持久化知识的真正正文。manifest 的设计刻意保持简单——它不是全文, 也不是严格索引——以便让 LLM 在召回时能够高效地进行第一轮筛选。topic 文件按语义主题命名, 并配以结构化的 front matter (name、description、type), 使得系统可以实现“先看目录、再读正文”的渐进式加载, 为后续的记忆召回与注入奠定了坚实的基础。下一章我们将深入探讨这些 topic 文件所承载的四种语义类型 (taxonomy) ——user、feedback、project 与 reference——以及它们各自的写入规范与正文结构要求。

3 四种语义类型详解

Claude Code 将自动记忆 (auto memory) 按内容语义强约束为四类: **user** (用户画像)、**feedback** (行为约束)、**project** (项目语境) 与 **reference** (参考指向)。这一分类体系定义在源码的 **memory-types.ts** 中, 决定了 memory 的写入策略、召回方式以及正文结构要求。本节逐一剖析这四类的定义边界与使用规范。

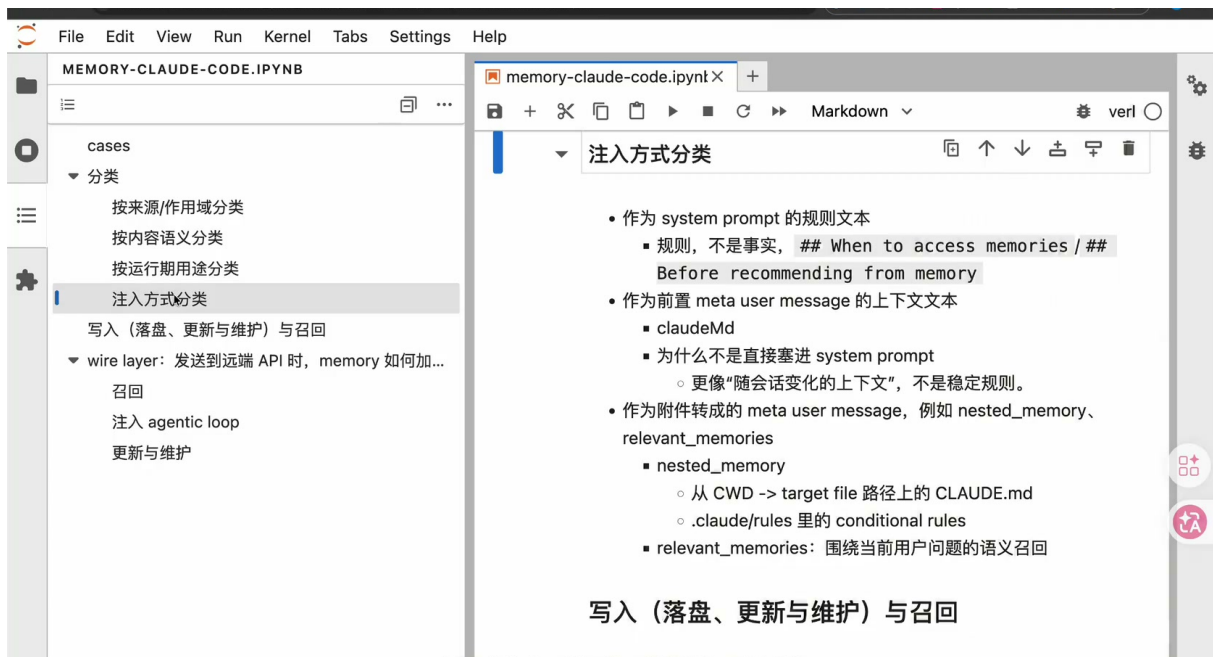


图 3: 四种语义类型的详细定义 (memoryTypes.ts) ³

3.1 User: 用户画像

定义: User memory 是关于“用户是谁”的个体式背景记忆，而非项目事实。它描述使用者的身份、偏好与协作风格，例如用户的职业背景（前端工程师、产品经理）、技术栈熟悉度、对解释深度的偏好等。

写入策略: 在每次 turn 结束后，extract memory agent 会基于完整对话提取用户画像信息，并将该 memory 标记为 `type: user`。写入时不会被混入 project 或 reference 类别的文件中。

召回特点: User memory 不会出现在 manifest (memory.md) 的生命清单中，它主要供模型内部 selector 判断用户本质，从而调整回答风格、解释力度和默认协作方式。由于其作用域跨越多个项目，它通常在全局或 user 作用域下存储。

正文结构要求: User memory 以事实性描述为主，记录稳定的人格/偏好特征，无需写成复杂规则，但应保持结构化（如分条列出用户的偏好）。

3.2 Feedback: 行为约束与规则

定义: Feedback memory 是对 agent 以后“该怎么做”的行为约束型记忆。它记录的不是“用户是谁”，而是“以后遇到类似情况时应该怎么做”。

Feedback 不只是负反馈

Feedback memory 既记录负反馈（如“不要 mock DB”），也记录正反馈（如“这个打包 PR 的方式是对的”）。很多初学者误以为 feedback 只保存纠错信息，实际上它是用来固化所有已验证的协作规则。

³视频画面时间区间：00:06:30–00:06:35。

写入策略：在对话中，用户明确表达”我喜欢/不喜欢”或 agent 的行为被纠正/肯定时，extract agent 会将其提炼为 **type: feedback** 的记忆。

召回特点：Feedback 记忆的召回与当前任务的行为模式强相关。当模型判断当前任务可能触发类似行为时，倾向于召回对应的 feedback 约束。

正文结构要求：不同于静态事实，feedback（以及 project）memory 的正文必须包含以下结构：

- **Rule / Facts：**核心规则或事实陈述；
- **Why：**为什么会有这条规则，帮助模型理解边界条件；
- **How to apply：**在何种场景下如何应用该规则。

这种结构化的目的是让记忆成为”可迁移规则”而非”静态事实”。有了 **Why**，模型才能避免机械套用；有了 **How to apply**，模型才能在正确场景下激活该规则。

3.3 Project：代码之外的现实

定义：Project memory 存储那些无法从代码库直接推导出来的项目上下文。它的核心是”非代码可推导”的外部现实。

Project memory 不是代码可推导的内容

Project memory 不是项目文档的大杂烩。如果某条信息可以通过阅读代码库直接推出（如函数签名、模块依赖），则不应写入 project memory。只有代码本身无法揭示的语境（如业务背景、截止日期、merge 冻结期）才值得保存。

写入策略：当对话中透露出代码文件无法体现的信息时（例如用户提到”本周五是 deadline”、”Q2 之后才能 merge”），extract agent 将其提炼为 **type: project** 的记忆。

召回特点：Project memory 在 **record / 解释阶段**最有价值。当用户问”为什么要做这个改动”、”最近这个方向有什么约束”时，代码本身往往不足以回答，而 project memory 提供了关键的外部语境。

正文结构要求：同 feedback 一样，project memory 也要求 **rule/facts + why + how to apply** 的结构，以便模型理解该语境的适用范围和触发条件。

3.4 Reference：去哪找事实

定义：Reference memory 不存储事实本身，而是存储”**去哪里找事实**”。它相当于项目中的”信息导航图”。

写入策略：当对话中提及外部系统及其用途时（例如”我们的任务在 Linear project X 中跟踪”、”Slack #backend-alerts 频道用于发布故障通知”），extract agent 将其写入 **type: reference** 的记忆。

召回特点：Reference 记忆在用户需要查找外部信息时被召回。它帮助模型节省搜索成本，直接指向正确的信息源，而不是在本地代码库中做无效搜索。

正文结构要求：每条 reference 记忆应清晰说明信息源的名称、URL/标识符、查询什么内容、以及该源头的更新频率或可信度（如有必要）。

3.5 四类 memory 对比

为了更直观地理解四类的侧重点，下面用表格对比它们的核心差异：

表 1: 四种语义类型 memory 的侧重点对比

维度 & User & Feedback & Project & Reference

定义 & 用户是谁 & 以后该怎么做 & 代码之外的现实 & 去哪找事实

写入触发 & 对话中暴露用户画像 & 行为被纠正/肯定时 & 出现非代码可推导的上下文 & 提及外部系统/信息源时

召回目的 & 调整风格与解释力度 & 约束未来行为模式 & 补充代码无法回答的项目语境 & 指向正确的外部信息源

结构要求 & 事实性描述 & Rule + Why + How & Rule + Why + How & 名称 + 位置 + 查询内容

常见示例 & 用户是前端工程师 & 不要 mock DB & 本周五是 deadline & Linear project X 看板

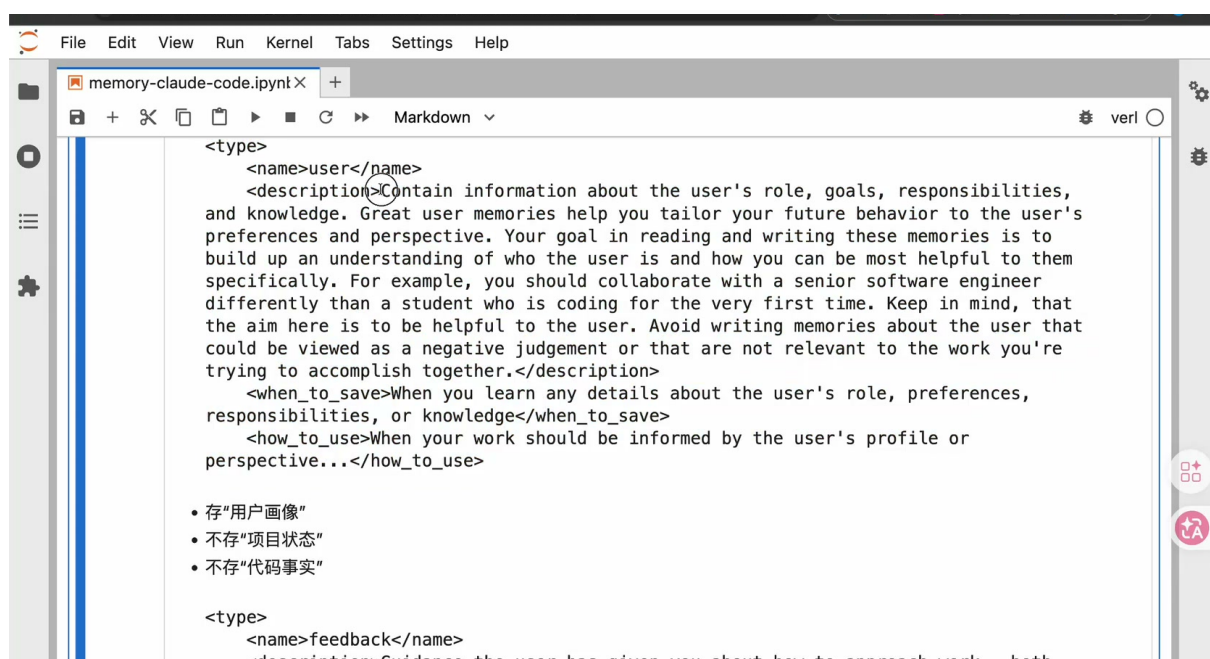


图 4: Memory type 的 XML 定义示例 (user / feedback)⁴

无论是哪一类 memory，运行时的统一处理流程都是相同的：扫描 front matter → 读取 description → selecting agent 判断是否相关 → 将相关内容注入 agent loop。

3.6 本章小结

本章深入解析了 Claude Code auto memory 的四种语义类型：

- **User**：刻画“用户是谁”，是个体层面的偏好与协作风格；
- **Feedback**：刻画“以后该怎么做”，既包含负反馈也包含正反馈，并需写成可迁移的规则结构；
- **Project**：刻画“代码之外的现实”，核心是“非代码可推导”的语境；
- **Reference**：刻画“去哪找事实”，是外部信息源的导航索引。

⁴视频画面时间区间：00:11:00–00:11:05。

这四类分类不仅决定了 memory 文件的命名与 front matter 标记，更关键的是塑造了 extract agent 的写入策略和 selecting agent 的召回逻辑。理解它们的语义边界，是设计合理的 memory 系统的前提。接下来，我们将进入第四章，具体看看 extract agent 和 selecting agent 是如何在运行时完成写入与召回的。

4 写入与召回机制

在了解了记忆的语义分类与 Manifest 结构之后，本节聚焦于 Memory 系统的两个核心运行环节：**写入（落盘）**与**召回（注入）**。Claude Code 并非让主模型直接操作文件，而是通过两个专用的子 agent——Extract Memory Agent 与 Selecting Memory Agent——分别负责 turn 结束后的持久化，以及基于 manifest 的语义召回。

4.1 Extract Memory Agent: turn 结束后的持久化

4.1.1 什么是 turn

在 Claude Code 的 agent loop 中，**turn** 是最基本的交互单元。

turn 的定义

一个 **turn** 指从用户发起 query 到 agent 形成最终 response 的完整一轮交互。期间 agent 可能进行多轮工具调用 (tool calls)，这些调用加在一起构成一个 turn，而不是每个工具调用单独算一轮。

换言之，用户提出一个问题，agent 内部经历了思考、调用工具、获得结果、再思考……最终给出一个 assistant response，这整个闭环才叫一个 turn。Extract Memory Agent 的触发时机，就放在每个 turn 结束之后，而非在工具调用的中间环节。

4.1.2 extract agent 的工作流程

写入与更新不是由主模型直接写文件，而是 **fork** 一个 **extraction sub-agent** 来执行总结和提取。其工作流程如图 5 所示：

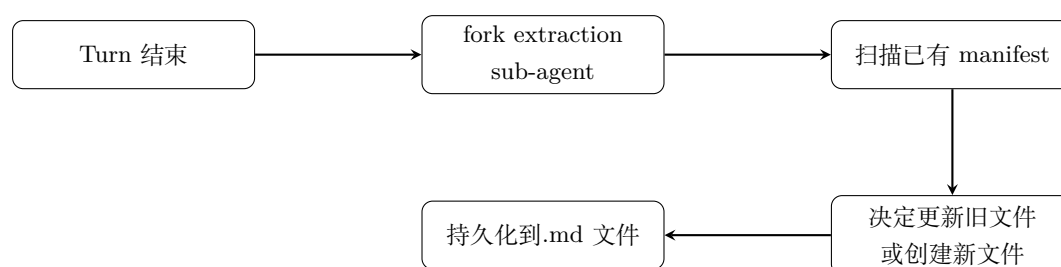


图 5: Extract Memory Agent 的工作流程（基于 00:14:19–00:15:38 画面内容绘制）

具体步骤如下：

1. **接收完整对话上下文**：sub-agent 拿到当前 turn 的全部内容，包括 user query、所有 tool calls / results 以及最终的 assistant response。

2. **扫描已有记忆**：系统先给 extract agent 一个 manifest（即轻量级的元数据清单），让它知道目前已有哪些 topic 文件、各自的主题与描述是什么。

3. **自主决定写或更新**：基于 manifest，LLM 判断本轮对话中产生的记忆应该：

- 归入某个已有 topic 并进行更新；还是
- 创建一个新的 topic.md 文件。

4. **限制权限并持久化**：sub-agent 的写权限受到约束，确保不会随意创建重复文件。

这种方式的好处在于：**写入行为与主推理解耦**，主 agent 不需要在回答用户的同时分心维护记忆文件；而是由一个专门负责总结的 sub-agent 在 turn 结束后统一处理，保证了记忆内容的完整性与一致性。

4.2 Selecting Memory Agent：基于 manifest 的语义召回

4.2.1 召回的本质

Selecting Memory Agent 的职责是**纯检索与排序**：它不开新工具、不执行代码，只基于 manifest 中各 topic 文件的名称与描述，判断哪些记忆与当前用户查询最相关，然后将选中的记忆内容注入到 agent loop 的上下文中。

4.2.2 召回的核心限制

在源码的 findRelevantMemories.ts 中，selecting agent 收到的 system prompt 对其行为施加了严格的约束。

Selecting Memory Agent 的召回限制

- **数量限制**：每次最多返回 5 个记忆文件。
- **确信原则**：仅包含你高度确信有用的记忆；若不确定，宁可返回空列表。
- **工具上下文重复避免**：如果当前对话中已经处于某工具的工作上下文（工具的 schema、tool call、tool results 已在 loop 中），则不再召回关于该工具的相关文档，防止冗余。

以下是从字幕内容中提炼出的 selecting system prompt 关键片段(对应视频中 00:16:01–00:17:38 的讲解)：

```
1 You are selecting memories that will be useful to respond to the user's request.
2
3 - Return at most 5 memory files.
4 - Only include memories you are highly confident will be useful.
5 - Base your decision on the memory topic file name and description.
6 - If you are not sure, do not include it in the list.
7 - Be very discerning.
8 - If no relevant memory files seem useful, return an empty list.
9
10 If a tool is already being used in the current conversation,
11 do not recall memory documents about that tool, because it is redundant.
```

Listing 5: Selecting Memory System Prompt 关键约束（提炼自 findRelevantMemories.ts）

4.2.3 限制背后的设计考量

- **最多 5 个**：防止上下文窗口被过多记忆淹没，保证主 agent 对用户当前问题的聚焦。
- **确信原则**：memory 中的信息可能已经过时（代码库在不断变化），如果相关性存疑，硬塞进去反而会造成幻觉或干扰。
- **避免工具上下文重复**：当 agent 已经在调用某个工具时，该工具的 schema、参数说明、返回结果已经存在于当前对话上下文中。此时再召回一份关于该工具的文档，不仅无增量价值，还会增加 token 消耗与 noise。

图 6 展示了从用户查询到记忆召回、再到注入 agent loop 的整体架构（对应视频 00:07:48–00:07:53 的画面）：

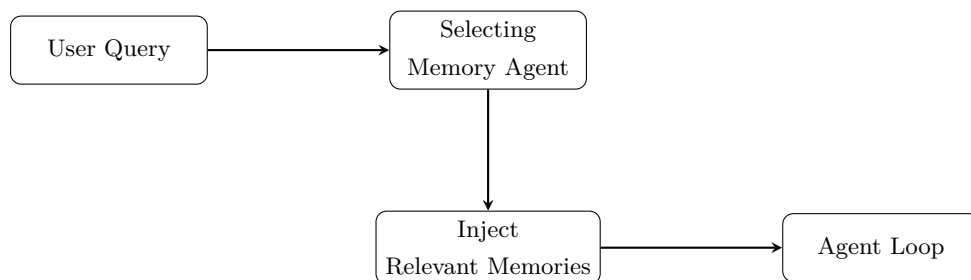


图 6: 记忆召回与注入架构示意（00:07:48–00:07:53）

4.3 本章小结

写入与召回构成了 Claude Code Memory 系统的运行时闭环：

- **写入侧**：每轮 turn 结束后，fork 一个 Extract Memory Agent，基于已有 manifest 决定更新旧文件或创建新文件。turn 的定义——从 user query 到最终 response 的完整交互单元——是理解这一时机的前提。
- **召回侧**：Selecting Memory Agent 执行纯检索排序，受限于“最多 5 个、确信原则、避免工具上下文重复”三大规则，确保注入 agent loop 的记忆既相关又精简。
- **整体设计**：通过子 agent 分工，将记忆的持久化与主推理解耦，同时以 manifest 为桥梁，在轻量级元数据层面完成召回决策，避免了直接对全部记忆正文做穷举匹配。

那么，当记忆被成功召回后，这些记忆以何种方式被注入 agent loop? system prompt 中又设置了哪些 memory policy 来约束模型的使用行为？这些问题将在第五章中展开讨论。

5 Prompt 注入、Memory Policy 与更新维护

5.1 Memory Policy 与 System Prompt

在 Claude Code 的 agent loop 中，memory 的加载并非简单地把文件内容塞进上下文，而是先通过 system prompt 向模型灌输一整套 **memory policy**——即关于“何时获取记忆、如何验证记忆、能否把记忆当事实”的行为约束。可以把 system prompt 里的 memory policy 理解为交通规则：它告诉模型哪些记忆可以直行注入，哪些记忆必须先停下来确认信号灯。

具体而言，system prompt 承担两类规则：

1. **召回前约束**：例如“最多召回 5 个 memory 文件”“只能选你高度确信有用的”“如果当前对话中已经处于某工具的工作上下文，就不要再召回该工具的文档，避免重复”。
2. **使用时约束**：memory 中提到的具体符号（函数、文件、flag）只能被视为“历史线索”，模型必须回到当前代码库重新读取并验证后，才能给出建议。

通过这种方式，system prompt 把 memory 从“被动知识库”提升为“受控的上下文资源”。它既保证模型能利用长期记忆，又防止模型不加甄别地直接引用过时信息。

5.2 幻觉防范：memory 不是 source of truth

在 coding agent 的语境下，memory 有一个天然缺陷：它不是实时同步的。auto memory 里的记录反映的是“写下那条记忆时的认知”，而代码库在此期间可能已经历了无数次提交、重构、分支合并甚至回滚。如果模型把 memory 当作绝对真相（source of truth），就会产生 **记忆幻觉**——例如坚信某函数仍然存在、某 flag 仍然有效，实际上它们早已被删除或改名。

如图 7 所示，源码中的 memory policy 明确写道：

memory 不是 source of truth。memory 只是后验线索。真正要给用户建议之前，必须回到当前代码/当前仓库中去验证。

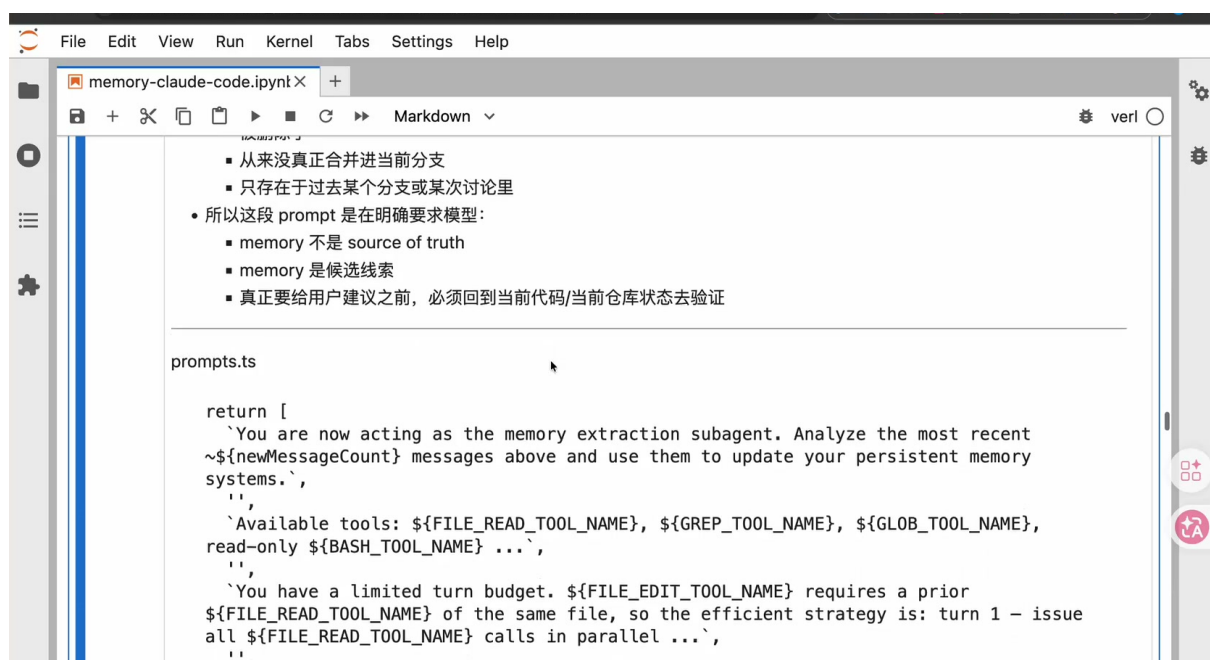


图 7: Memory Policy Prompt: memory 不是 source of truth⁵

这背后的核心哲学可以概括为三层：

- **时间滞后性**：memory 的写入发生在 turn 结束后，而代码的修改可能发生在任意时刻；两者之间没有时间锁。
- **来源有限性**：memory 来源于某次对话或某次观察，它可能只记录了局部上下文，甚至只是讨论中的假设，而非已合并到主分支的事实。

⁵视频画面时间区间：00:16:03–00:16:08。

- **可变性**：函数、变量、配置项随时可能被重命名、迁移或删除；memory 里的符号名称如果不经验证，就成了“幽灵引用”。

记忆幻觉的风险与防范

风险：模型可能在未验证的情况下，直接基于记忆中的函数签名或文件路径生成代码修改、测试建议，导致编译失败或逻辑错误。

防范措施：在 system prompt 中强制声明“memory 不是 source of truth”，并在 agent loop 的执行策略里要求：任何基于 memory 的具体符号操作前，必须通过 Read、Grep 等工具重新确认当前代码库状态。简言之——**先验证，再行动**。

5.3 注入方式：system prompt、user context 与 nested memory

当相关记忆被召回后，Claude Code 采用多种注入方式将其送入 agent loop。不同层级的注入方式对应不同的角色与信任度：

1. **System Prompt 规则注入**：这部分注入的不是记忆事实本身，而是“如何对待记忆”的元规则。例如 memory policy、when to retrieve、如何验证符号等。因为 system prompt 权重高且稳定，适合放置行为约束型规则。
2. **前置 User Context (Meta User Message)**：在真实用户 query 之前，系统会插入一条由 agent 构造的 user message，里面可以包含 `claude.md`（或项目对应的上下文文档）、用户画像、以及从 memory 中召回的 relevant memories。这种注入方式的好处是：它以“上下文摘要”的形式出现，不抢占 system prompt 的指令位，同时让模型在正式回答前就能读取到相关线索。
3. **附件型 / Nested Memory**：对于某些结构化召回结果（例如从 `memory.md` 索引中定位到的具体 topic 文件），系统可能将其以嵌套（nested）附件的形式注入。这类注入更贴近“事实层”，通常携带文件名、摘要描述和关键规则片段，方便模型在后续工具调用中按需展开。

这三种方式构成了一条由“规则 $\rightarrow$ 上下文 $\rightarrow$ 事实”的层级链路：system prompt 设定行为的边界，user context 提供当前会话所需的综合背景，nested memory 则补充具体的知识点。三者协同，既避免了 system prompt 过度膨胀，也保证了记忆内容能被模型有效利用。

5.4 更新维护机制：extract sub-agent 的调用时机与 system prompt 继承

auto memory 的写入、更新与维护是一个闭环，统一由 **extract memory sub-agent** 负责。它的调用时机设计得非常明确：**每轮 turn 结束后**。所谓一个 turn，是指从用户发出 query 到 agent 完成所有工具调用、形成最终 assistant response 的完整交互周期。turn 一结束，主 agent 就会 fork 出一个 extraction sub-agent，把这轮对话的全貌作为输入，分析是否需要从对话中提取出新的持久化记忆。

该子 agent 的工作流程可以用图 8 的关键步骤概括：先扫描已有 manifest（即 `memory.md` 中的轻量级目录摘要），获得当前已存在的 topic 清单；然后基于本轮对话的内容，自主判断是应该**更新旧文件**还是**创建新文件**。整个决策过程是语言模型驱动的，而非硬编码的模板匹配。

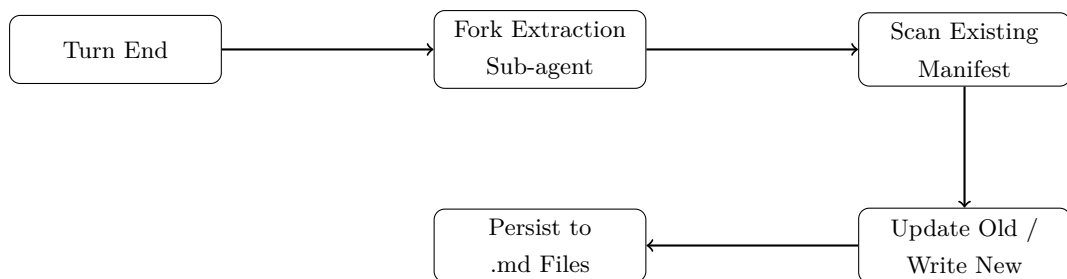


图 8: 自动写入与更新维护机制示意⁶

值得注意的是，这个 extraction sub-agent 并非从零开始执行；它会**继承副会话的完整 system prompt**。这意味着，提取、总结、更新记忆的过程同样要遵守主 agent 的 memory policy 和幻觉防范规则。system prompt 的继承确保了子 agent 在判断哪些内容值得写入、如何组织 rule + facts + why/how 结构时，与主 agent 保持一致的策略与价值观。

设计启示

将写入/更新逻辑交给一个独立的 sub-agent 处理，有以下好处：

- **解耦主 loop**：主 agent 专注完成用户任务，不必在每次 turn 中分心维护长期记忆。
- **充分利用完整上下文**：sub-agent 能回顾整个 turn 的用户 query、所有工具调用与结果、以及最终回答，做出更全局的提取决策。
- **保持策略一致性**：通过继承 system prompt，子 agent 不会偏离主 agent 的记忆分类、幻觉防范与写入规则。

5.5 本章小结

本章围绕 Prompt 注入、Memory Policy 与更新维护三个核心维度，剖析了 Claude Code 如何保证记忆既能被有效召回，又不会被盲目采信。关键要点如下：

- **System Prompt 是控制阀**：通过 memory policy 约束模型的召回与使用行为，设定“上限 5 条”“不确定则不召回”等硬性规则。
- **Memory 不是 source of truth**：代码库持续演化，而记忆只是历史后验线索。任何基于记忆的具体操作建议，都必须先回到当前仓库验证。
- **注入方式分层**：system prompt 负责规则，user context 负责上下文摘要，nested memory 负责具体知识点，三者协同构成完整的记忆支持。
- **更新维护由 sub-agent 闭环**：extract memory sub-agent 在每轮 turn 结束后 fork 执行，继承主 agent 的 system prompt，基于 manifest 决定更新旧文件或创建新文件，实现记忆的自动演进与一致性维护。

6 总结与延伸

通过对 Claude Code 源码的分析，我们可以勾勒出一个完整的现代 agent 记忆系统闭环：

⁶视频画面时间区间：00:18:58–00:19:03。

- **语义分类(taxonomy)**: 所有自动维护的长期记忆被强约束为 user、feedback、project、reference 四类。分类不是装饰，而是决定了提取策略、召回方式与正文结构。
- **落盘结构(manifest + topic.md)**: memory.md 作为轻量级的 manifest，只记录 front matter 层面的元数据摘要；真正持久化的知识正文下沉到各个 topic.md 文件中，实现”先看目录、再读正文”的渐进式加载。
- **写入与召回机制**: 每轮 turn 结束后 fork 的 extract memory agent 负责写入与更新维护；而 selecting memory agent 基于 manifest 执行语义召回，每次最多返回 5 个高度相关的记忆文件，避免上下文被淹没。
- **注入与幻觉防范**: system prompt 中的 memory policy 设定了明确的约束规则——memory 不是 **source of truth**，任何具体符号都必须在回到当前代码库验证后才能采信。相关记忆通过 system prompt 规则、前置 user context 或 nested memory 等方式分层注入 agent loop。

这套设计对我们构建自己的 coding agent 具有通用启示：在动手落盘之前，**先定义好记忆的语义分类体系**；在解决召回效率之前，**先设计清晰的 manifest 与 topic 文件结构**；在让模型使用记忆之前，**先在 prompt 层面建立严格的幻觉防范与验证策略**。

若想进一步深入研究源码，建议重点关注以下文件：`memory-types.ts` (分类定义)、`extractMemories.ts` 及相关子 agent prompt (写入与更新机制)、`findRelevantMemories.ts` (召回与 selecting 策略)、以及 `prompts.ts` 中的 memory policy (幻觉防范规则)。

6.1 本章小结

Claude Code 的 Memory 系统证明了：一个真正可用的长期记忆，不只是”把对话存进文件”那么简单，而是一个围绕分类、落盘、召回、注入与保守验证的完整工程体系。只有把这些环节串联成闭环，agent 才能在持续演进的项目环境中，既”记得住”，又不”记错”。